

DARK HOURS

CASE STUDY: Procedural Environment Systems & Gameplay Flow

"Designing scalable procedural gameplay spaces for a cooperative horror game with strong replayability, readability, and pacing constraints."

A Technical Design case study by Esteban Marques, Associate Lead Game Designer on Dark Hours

What is Dark Hours?

Dark Hours is a co-op survival "comedy-horror" game where up to 4 players venture in procedurally generated levels to collect loots and complete various objectives all while being hunted down by a fearsome entity. It was released in early access after 6 months of production and was constantly updated with regular and major content updates for a year.

+500k
Copies on steam



A Year of Early Access
With 5 major Content Updates

Introducing the case study

Developed on **Unreal Engine** in about one and a half year with one year being dedicated to fleshing out the game's content and mechanics during the **early access**, I've worked on Dark Hours as an **Associate Lead Game Designer** with a team of less than 10 people.

While my work on this project was broad, this case study will focus on my work on the procedurally generated level design.

Our goals for the procedurally generated levels system

- Create varied level layouts while maintaining a **small production scope**
- Preserve **gameplay readability** during the most stressful gameplay situations
- Support multiple objective types across all generated levels
- Maintain consistent **pacing and navigation flow** despite procedural layouts
- Allow **rapid iteration** for both design and art integration

To reach these goals, I worked on a custom room population system to use in tandem with our level generation algorithm and make sure our levels were varied while maintaining a coherent flow and a readable navigation.

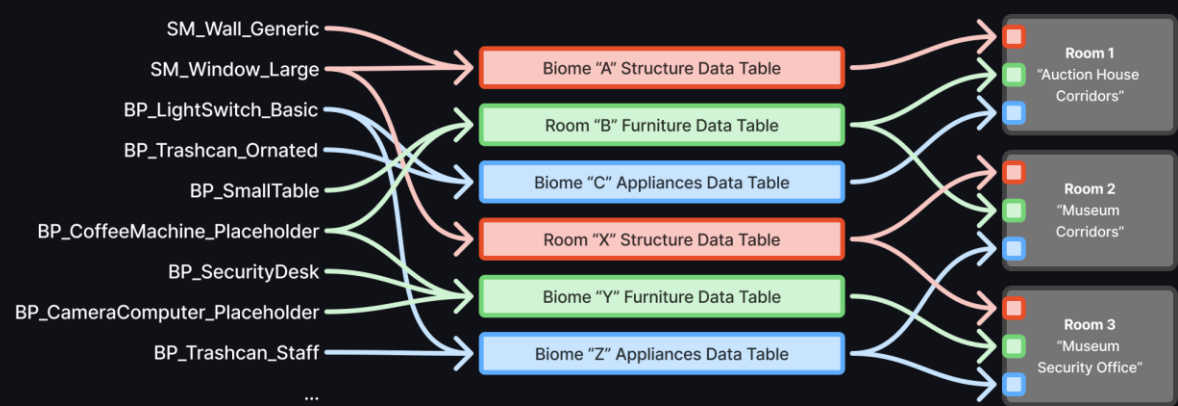
Creating the Procedural Room Population System

To support replayability while maintaining a small production scope, I designed and implemented a procedural room population workflow using Unreal Engine's experimental PCG Framework, Blueprint systems, and Data Tables. The system allowed the generation of fully playable rooms from simple volumes authored by the level generation algorithm.

Key Features:

- Data-driven room population using Data Tables
- Modular room categories and biome variations
- Procedural placement of gameplay objects, furniture, loot, and navigation elements
- Rapid greybox iteration before final art integration

To accelerate production and iteration, I created greyboxed modular room elements that could quickly be tested in-game before being replaced by final art assets. This workflow allowed us to validate gameplay readability, navigation, and object density early in production.

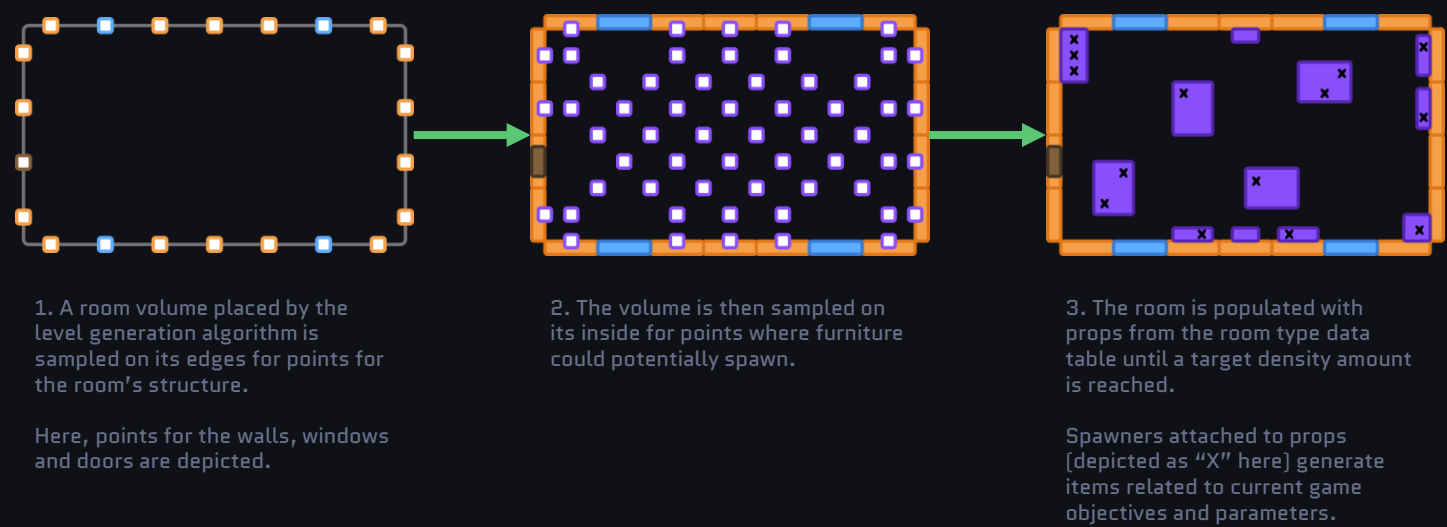


Dark Hours is a co-op survival “comedy-horror” game where up to 4 players venture in procedurally generated levels to collect loots and complete various objectives all while being hunted down by a fearsome entity. It was released in early access after 6 months of production and was constantly updated with regular and major content updates for a year.

Because room types and asset pools were **fully data-driven**, we could rapidly iterate on room identities, pacing rules, and gameplay layouts without requiring major technical changes.

Generation Process Explained

Unreal Engine’s experimental PCG Framework was used to sample the room volumes generated by our level generation algorithm. Sampled points from the volumes were then used as the base to generate every asset that makes up a room in Dark Hours.



Gameplay Flow & Level Rules

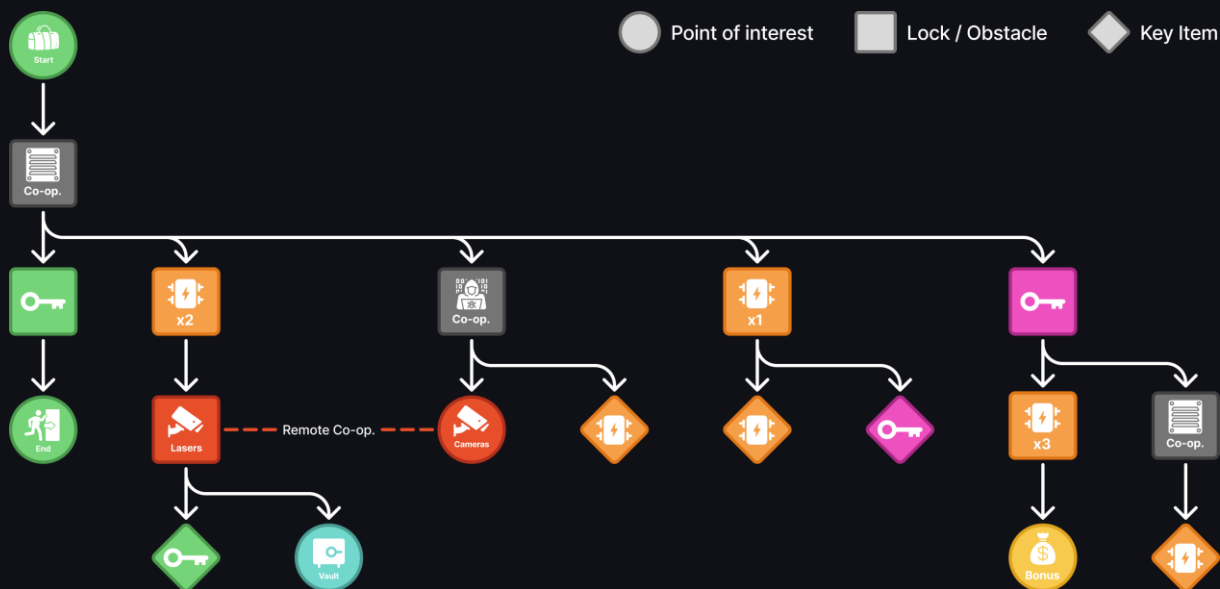
One of the main design challenges was balancing procedural randomness with intentional gameplay pacing.

The generated levels needed to feel unpredictable enough to support replayability while still maintaining readable navigation, cooperative coordination, and consistent progression flow. To achieve this, I designed flowcharts and progression guidelines describing how players should progress through a typical level.

These guidelines helped define:

- Critical path structure
- Dead-end distribution
- Key item requirements
- Loot risk/reward placement
- Navigation readability under stress
- Objective pacing across the level

Here's how a flowchart looked at some point in development:



I then collaborated closely with programmers to implement parameters influencing the generation algorithm so generated layouts would consistently follow the designed flow rules.

Having these systems represented visually through flowcharts also made iteration significantly faster and improved communication across disciplines, especially for non-design team members.

Objective Integration

Because Dark Hours featured multiple objective types, every procedurally generated level needed to remain compatible with a wide variety of gameplay scenarios.

Objectives included:

- Loot collection
- Targets destruction
- Timed explosive placement
- Item Retrieval

To support this flexibility, I designed a modular tagging system allowing objective spawners and gameplay locations to be categorized according to gameplay needs, room types, visibility, and progression requirements. This allowed objectives and loot to be dynamically distributed while still preserving intentional gameplay readability and pacing.

Spawner placement inside room modules was carefully authored to control player visibility and exploration incentives.

As a result, the same level structure could support **many different gameplay scenarios** while remaining readable and **intentionally designed**.

Challenges & Iteration

One of the project's biggest technical challenges came from using Unreal Engine's experimental PCG Framework relatively early in its development lifecycle. Documentation was limited, many workflows had to be discovered through experimentation, and the system occasionally introduced unstable behaviors that required constant iteration and debugging alongside programmers.

At the same time, procedural generation created an important design challenge: **maintaining authored gameplay pacing inside unpredictable layouts**.

Too much randomness could negatively impact navigation clarity, cooperative communication, tension buildup and objective readability. A large part of the iteration process therefore focused on finding the right balance between procedural variety and intentional level design.

Outcome & Lessons Learned

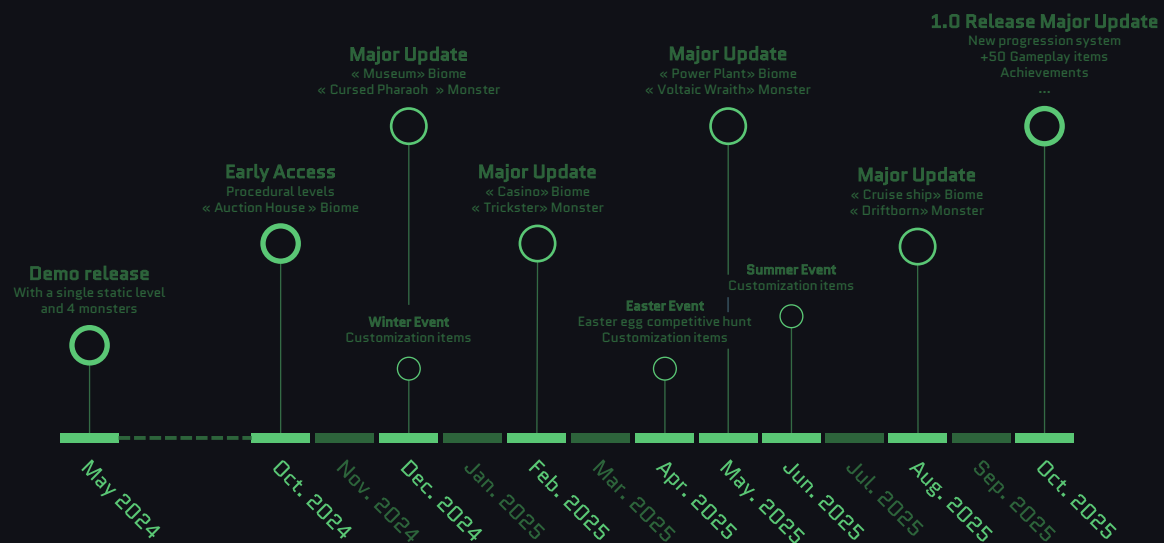
The procedural generation workflow became a **core** pillar of what made Dark Hours' success, shipping with the Early Access release and remaining stable throughout a full year of major content updates, proving that the data-driven approach was the right foundation from the start.

As the game evolved, the system was extended to support features like multi-floor layouts, high-ceiling rooms, suspended platforms crossing rooms from upper floors and more. All of this was integrated without requiring a structural overhaul, simply by expanding the existing data tables and PCG rules.

On a personal level, working with an experimental framework that had limited documentation taught me to treat uncertainty as part of the design process. Building robust, iterable tooling from the start rather than optimizing too soon turned out to be the decision that paid off most in the long run.

If I were to approach a similar system again, I would invest earlier in debugging tools for the generation process. A lot of iteration time was spent diagnosing unexpected outputs that better in-editor feedback could have surfaced much faster.

Dark Hours' Roadmap from Demo to 1.0 Release



Procedurally populated rooms in different biomes

